

Blockchain-Based Environmental Data Integrity for Distributed Monitoring Systems

KM

June 24, 2026

Abstract

Distributed environmental monitoring systems produce time-series records that may pass through multiple sensors, gateways, storage systems, correction workflows, and publication channels. This creates an information-systems integrity problem: users need to determine whether records were modified, deleted, replayed, inserted, corrected without authorization, or disconnected from provenance evidence. This paper evaluates a lightweight blockchain-inspired proof-of-concept for environmental data integrity. Using a bounded OpenAQ API v3 dataset covering Warsaw, Berlin, Paris, and Madrid from 2025-07-01 to 2025-12-31, the study compares four integrity models: conventional storage, audit trail, audit trail with hash-chain linkage, and audit trail with hash-chain linkage plus provenance and permission reconstruction. The experiment uses 112,973 canonical records and 25 controlled tampering scenarios. The resulting threat-coverage matrix shows 20 detected scenarios and 5 expected non-detections, with no missed, partial, or unexpected-alert cases in the implemented scenario set. The contribution is not an environmental air-quality analysis or a production blockchain deployment, but a reproducible information-systems evaluation of how audit trails, hash chains, and provenance reconstruction affect verification coverage for distributed monitoring records.

1 Introduction

Environmental monitoring systems increasingly rely on distributed sensing infrastructures to collect observations about air quality, water quality, meteorological conditions, and other environmental parameters. Such data support public health assessment, environmental policy, infrastructure planning, agricultural decisions, and local risk management. `TODO:CITATION.NEEDED` From an information-systems perspective, however, the reliability of environmental data depends not only on sensor accuracy, but also on the integrity of the complete data lifecycle: acquisition, transmission, storage, correction, aggregation, publication, and reuse.

Environmental sensor networks introduce several data integrity challenges. Measurements may be collected by geographically dispersed devices, transmitted through intermittent communication links, buffered locally before synchronization, processed by different organizations, or integrated into public and semi-public datasets. `TODO:CITATION.NEEDED` These conditions are common in distributed monitoring settings, including but not limited to rural deployments, multi-station public networks, and heterogeneous monitoring infrastructures. Under such conditions, it may be difficult to determine whether a measurement record is original, complete, delayed, corrected, replayed, removed, or modified after collection.

The trustworthiness of environmental data therefore requires more than conventional storage. It requires provenance information that describes where a record originated, which sensor or gateway produced it, when it was collected, which transformations or corrections were applied, and who or

what system authorized those changes. `TODO:CITATION_NEEDED` Without such provenance, downstream users may be unable to distinguish between valid corrections, technical failures, missing observations, and unauthorized manipulation. This problem is especially important when environmental data are used for regulatory reporting, infrastructure monitoring, scientific analysis, or public communication. `TODO:CITATION_NEEDED`

Blockchain-inspired audit trails offer one possible approach to this problem. The relevant idea is not cryptocurrency, tokenization, or fully public consensus. Rather, the useful properties are append-only event recording, cryptographic linking of records, timestamped transactions, hash-based verification, and explicit representation of permissions and corrections. `TODO:CITATION_NEEDED` A lightweight blockchain-inspired architecture can store raw environmental measurements off-chain while preserving hashes, audit events, provenance metadata, and verification evidence in an integrity layer. Such a design may support later detection of unauthorized modification without requiring every environmental dataset to be stored directly on-chain.

This direction is motivated by data integrity practices from regulated environments, including clinical research, laboratory systems, and other domains where records must remain attributable, legible, contemporaneous, original, accurate, complete, consistent, enduring, and available. `TODO:CITATION_NEEDED` In these environments, audit trails, controlled corrections, source data verification, role-based authorization, and traceable data custody are treated as core components of trustworthy data management. Although environmental monitoring systems are governed by different technical and legal requirements, the underlying integrity problem is related: collected observations must remain verifiable across organizational, technical, and temporal boundaries.

The research gap addressed in this work is the lack of a clearly defined and experimentally testable integrity framework that compares conventional storage, audit trails, hash chains, and provenance-aware verification for distributed environmental monitoring records. Existing work on environmental sensor networks often focuses on sensing, communication, calibration, data quality, or analytics, while blockchain-oriented environmental systems often emphasize decentralization or transparency. `TODO:CITATION_NEEDED` Less attention has been given to environmental data integrity as a threat-coverage problem involving source record identity, timestamped provenance, controlled correction, hash verification, permission changes, audit review, and data custody from collection to publication. `TODO:CITATION_NEEDED`

The primary research question is:

How do increasingly expressive integrity models compare in their ability to detect and explain threats to distributed environmental monitoring records?

To answer this question, the study pursues the following objectives:

1. Define a threat model for distributed environmental monitoring records.
2. Compare four integrity models: conventional storage, audit trail, audit trail with hash chain, and audit trail with hash chain plus provenance and permission reconstruction.
3. Propose a lightweight blockchain-inspired architecture that separates raw environmental measurements from hashes, audit metadata, permission events, and verification records.
4. Specify verification workflows for value modification, timestamp modification, record deletion, fake insertion, replay, broken provenance, unauthorized correction, revoked key usage, missing correction reasons, and delayed synchronization.

5. Evaluate the models through a reproducible proof-of-concept using public environmental monitoring data and controlled tampering scenarios.

The expected contribution of this paper is a reproducible information-systems evaluation of blockchain-inspired integrity mechanisms for distributed environmental monitoring data. The contribution is intentionally scoped: it does not claim that blockchain technology improves environmental outcomes directly, nor does it propose a production blockchain network. Instead, it examines how audit trails, hash chains, provenance metadata, and permission reconstruction affect the detectable threat coverage of environmental records whose trustworthiness may otherwise be difficult to establish.

2 Related Work

2.1 Environmental Monitoring Systems

Environmental monitoring systems collect observations about air quality, water quality, meteorological conditions, soil conditions, noise, radiation, and other parameters relevant to environmental assessment and infrastructure governance. `TODO:CITATION_NEEDED` These systems may combine fixed monitoring stations, mobile sensors, laboratory measurements, remote sensing sources, and public data repositories. `TODO:CITATION_NEEDED` Their outputs are often used by public authorities, researchers, infrastructure operators, and local communities to support environmental decisions and risk assessment. `TODO:CITATION_NEEDED`

The literature on environmental monitoring commonly emphasizes sensing accuracy, spatial coverage, calibration, communication protocols, data quality, and analytics. `TODO:CITATION_NEEDED` These topics are essential, but they do not fully address the integrity of the data lifecycle after a measurement has been produced. In particular, the questions of who or what produced a record, whether it was later corrected, whether it was transmitted without alteration, and whether its provenance can be independently verified require additional attention.

2.2 Distributed and Rural IoT Monitoring

Distributed IoT monitoring systems operate under constraints that differ from centralized database environments. Monitoring locations may be separated geographically, maintained by different organizations, connected through heterogeneous infrastructure, or synchronized at different times. Rural deployments can intensify these constraints through sparse connectivity, limited access to maintenance personnel, long distances between devices, heterogeneous energy availability, and weaker institutional infrastructure for continuous supervision. `TODO:CITATION_NEEDED` These constraints affect not only data availability, but also the ability to reconstruct the history of environmental records when anomalies, delays, or missing values occur.

IoT-based monitoring is frequently discussed in terms of low-cost sensing, wireless communication, edge processing, gateway architectures, and energy-efficient data collection. `TODO:CITATION_NEEDED` For the present study, the important issue is how such deployments can preserve data trustworthiness when measurements are buffered locally, synchronized later, or handled by multiple technical and organizational actors. This creates a need for integrity mechanisms that are lightweight enough for constrained deployments but expressive enough to support later audit and verification.

2.3 Environmental Sensor Networks

Environmental sensor networks provide the technical basis for many modern monitoring systems. They typically consist of sensor nodes, communication links, gateways, storage backends, and analytical services. `TODO:CITATION_NEEDED` Prior research has examined network design, sensor placement, calibration, data fusion, anomaly detection, and quality assurance for environmental measurements. `TODO:CITATION_NEEDED`

However, environmental sensor networks also create data lineage challenges. A measurement may pass through a sensor, local controller, gateway, cloud service, preprocessing script, database, and public reporting interface before it is used. Each step may introduce transformations, aggregation, correction, or filtering. `TODO:CITATION_NEEDED` Without an explicit record of these steps, later users may be unable to distinguish original observations from derived or corrected records.

2.4 Data Provenance in Environmental Data

Data provenance describes the origin, history, custody, transformation, and context of a data item. `TODO:CITATION_NEEDED` In environmental monitoring, provenance can include the source station, sensor identifier, location, parameter, timestamp, unit, calibration state, processing step, responsible organization, and publication history. `TODO:CITATION_NEEDED` Provenance is especially important when environmental records are reused outside the original collection context, for example in scientific analysis, public reporting, compliance workflows, or infrastructure planning.

Existing provenance approaches can document data lineage, but they may not always provide tamper-evident verification of the recorded lineage itself. `TODO:CITATION_NEEDED` For the present work, provenance is treated as both descriptive metadata and an audit object: it should be possible to verify whether provenance records have been altered, whether a correction refers to a valid original record, and whether the actor performing a change was authorized at the time of the event.

2.5 Data Integrity and Trustworthiness

Data integrity refers to the preservation of data accuracy, completeness, consistency, and resistance to unauthorized modification across the data lifecycle. `TODO:CITATION_NEEDED` In environmental monitoring, integrity concerns include altered measurements, missing records, duplicated or replayed observations, inconsistent timestamps, undocumented corrections, and loss of custody metadata. `TODO:CITATION_NEEDED`

Trustworthiness is broader than integrity alone. It also includes transparency about collection methods, sensor limitations, calibration status, processing history, and governance responsibilities. `TODO:CITATION_NEEDED` A monitoring system can produce technically accurate data while still being difficult to trust if users cannot reconstruct how a record was created and changed. This motivates an audit-oriented approach in which integrity checks, provenance metadata, and correction history are treated as first-class elements of the monitoring architecture.

2.6 Blockchain for Environmental Systems

Blockchain and distributed ledger technologies have been proposed for a range of environmental and sustainability applications, including environmental data sharing, carbon accounting, supply-chain transparency, resource management, and sensor data verification. `TODO:CITATION_NEEDED` In these contexts, the most relevant technical properties are append-only recording, cryptographic linking, distributed verification, and resistance to undetected modification. `TODO:CITATION_NEEDED`

For distributed environmental monitoring, a fully public or fully decentralized blockchain may be unnecessary or impractical. `TODO:CITATION_NEEDED` The present study therefore focuses on blockchain-inspired architecture rather than cryptocurrency-oriented deployment. The core idea is to use hash-linked events, timestamped records, permission events, and verification reports to preserve evidence of data integrity while keeping raw environmental measurements in conventional off-chain storage.

2.7 Audit Trails

Audit trails record actions performed on data and systems, including creation, modification, review, approval, correction, deletion attempts, and access-control changes. `TODO:CITATION_NEEDED` In regulated environments, audit trails support accountability by recording who performed an action, when it occurred, what object was affected, and what changed. `TODO:CITATION_NEEDED`

Environmental monitoring systems can benefit from similar audit concepts. A measurement correction, sensor replacement, data invalidation, delayed upload, or publication event can be represented as an audit event rather than an invisible change to a database row. This approach is especially relevant when multiple actors participate in the data lifecycle, including sensor operators, local authorities, laboratories, environmental agencies, and researchers. `TODO:CITATION_NEEDED`

2.8 Hash Verification

Hash verification provides a lightweight mechanism for detecting whether data have changed since a reference digest was computed. `TODO:CITATION_NEEDED` When records are serialized deterministically and linked through previous hashes, later verification can detect changes to record values, timestamps, metadata, or event order. `TODO:CITATION_NEEDED` Hash verification is therefore suitable for proof-of-concept experiments involving controlled tampering and known ground-truth labels.

Hashing alone does not solve all data trust problems. It cannot determine whether an originally submitted measurement was physically correct, whether a sensor was calibrated, or whether an authorized actor entered false information. `TODO:CITATION_NEEDED` Its value lies in detecting unauthorized or undocumented changes after a record has been committed to an integrity layer. For this reason, the proposed architecture combines hash verification with provenance metadata, audit events, and permission-state reconstruction.

2.9 Literature Gap

The reviewed topic areas suggest three gaps that motivate this study. First, environmental monitoring and sensor-network literature provides extensive treatment of measurement, communication, and data quality, but less explicit treatment of data integrity as an auditable lifecycle problem. `TODO:CITATION_NEEDED` Second, provenance literature supports lineage description, but the relationship between provenance metadata and tamper-evident verification remains underdeveloped for distributed monitoring workflows. `TODO:CITATION_NEEDED` Third, blockchain applications in environmental systems often emphasize decentralization or transparency, while fewer studies focus on lightweight audit-trail architectures evaluated through explicit threat-coverage matrices. `TODO:CITATION_NEEDED`

This paper addresses these gaps by proposing and evaluating a blockchain-inspired integrity architecture that separates raw environmental measurements from hash records, provenance metadata, permission events, correction history, and verification reports. The intended contribution

is not a production blockchain network, but a structured and experimentally testable model for environmental data integrity in distributed monitoring systems.

3 Methodology

3.1 Study Design

The study evaluates data integrity mechanisms for distributed environmental monitoring records. The environmental domain is used as a realistic information-systems substrate: monitoring data are time-series based, multi-source, provenance-sensitive, and often reused outside their original collection context. The study does not evaluate air-quality correctness, pollutant behavior, sensor calibration, or environmental policy effects.

The experiment compares four increasingly expressive integrity models under controlled tampering scenarios. Each model is built from the same canonical dataset, and each applicable threat scenario is injected into the corresponding model artifact. A verifier then examines the tampered artifact and produces alert codes. The evaluation compares verifier alerts against scenario labels and aggregates the results into a threat-coverage matrix.

3.2 Dataset

The proof-of-concept uses a bounded extract from OpenAQ API v3. `TODO:CITATION_NEEDED` The dataset identifier used in the experiment is `openaq.capitals.2025_h2`. The selected time window is from 2025-07-01 to 2025-12-31. Four European capital cities were included: Warsaw, Berlin, Paris, and Madrid. For each city, three monitoring locations were selected, producing a total of 12 monitoring locations.

The station selection creates a distributed, multi-source environmental monitoring dataset while keeping the experiment bounded and reproducible. The current experiment treats all selected stations as one combined canonical dataset and one global event stream per integrity model. It does not evaluate cities or stations independently, and it does not perform cross-sensor environmental plausibility checking.

The preprocessing step normalized raw OpenAQ records into canonical measurements. The input contained 116,361 OpenAQ records. After preprocessing, 112,973 canonical records remained and 3,388 records were dropped because the measurement value was missing. The final canonical dataset contained nine measured parameters.

3.3 Integrity Models

Model A represents conventional storage only. It stores canonical measurement records without an audit trail, hash-chain linkage, provenance state, or permission state. Its verifier is limited to schema checks and duplicate record identifier checks.

Model B represents an audit trail without hash-chain linkage. Measurements are represented as audit events with deterministic payload hashes and event identifiers. This enables local event-integrity checks, but the event stream is not linked by previous-hash references.

Model C extends Model B with hash-chain linkage. Each event contains a payload hash, a previous hash, and a block hash. This model supports verification of event order and chain continuity.

Model D extends Model C with provenance and permission reconstruction. It includes actor identifiers, key identifiers, permission events, revocation events, correction events, and active key-

state reconstruction. This model supports additional checks for actor-related and governance-related integrity failures.

3.4 Threat Scenarios

The base scenario set covers five data and stream-integrity threats: value modification, timestamp modification, record deletion, fake record insertion, and replay. These scenarios are applied to Models A-D where the model representation supports the scenario.

Five additional scenarios are specific to Model D: broken provenance, unauthorized correction, revoked actor key usage, missing correction reason, and delayed synchronization. These scenarios require provenance, permission, correction, or synchronization metadata and are therefore outside the implemented scope of Models A-C.

Each implemented scenario uses one deterministic injection per applicable model/threat pair. The current design is a reproducible mechanism-comparison benchmark, not a statistical robustness evaluation over repeated random injections.

3.5 Evaluation Semantics

The verifier output is evaluated using scenario labels. A scenario receives the status `detected` when the expected alert code is present in the verifier output. A scenario receives `expected_not_detected` when the threat is intentionally outside the detection capability of the evaluated model. A `not_applicable` cell indicates that the scenario is outside the implemented scope of a model.

The current evaluation reports scenario status counts and a threat-coverage matrix. It does not report precision, recall, F1 score, false-positive rate, or false-negative rate because explicit negative cases were not defined in this experiment.

4 System Architecture

4.1 Proof-of-Concept Scope

The proof-of-concept is intentionally lightweight. Its purpose is to support the experimental comparison of integrity models rather than to implement a production blockchain platform. The implementation uses Python-based data processing, tabular preprocessing, JSON or JSONL artifacts, SQLite storage where useful, deterministic hashing, controlled tampering generation, model-specific verification, and metrics aggregation.

The system keeps raw environmental measurements outside the integrity layer. Integrity evidence is represented through canonical records, audit events, payload hashes, hash-chain links, provenance metadata, permission events, and verification reports. This separation reflects the central design choice of the paper: the integrity layer should make record manipulation visible without requiring environmental measurements to be stored directly on-chain.

4.2 Data Flow

The data flow begins with a bounded OpenAQ extract. Raw records are normalized into canonical measurement records with stable identifiers and standardized fields. The same canonical records are then transformed into four model artifacts. Model A stores conventional measurement records. Model B stores audit events. Model C stores hash-linked audit events. Model D stores hash-linked events enriched with provenance and permission state.

For each applicable threat scenario, a tampering generator creates a modified artifact and a scenario label. The verifier reads the tampered artifact, recalculates relevant hashes, reconstructs stream or permission state where required, and emits alerts. The evaluator compares these alerts against the expected scenario labels and exports per-scenario metrics, aggregate status counts, and the threat-coverage matrix.

4.3 Hashing Strategy

The proof-of-concept uses deterministic JSON serialization and SHA-256 hashing for the integrity artifacts. `TODO:CITATION_NEEDED` Payload hashes are computed from canonicalized event payloads. In Models C and D, each event also contains a previous-hash reference and a block hash, allowing the verifier to detect disrupted chain continuity.

The hash strategy is used for experimental reproducibility and tamper evidence. It is not presented as a final cryptographic standard selection for production environmental monitoring systems. Any production recommendation about algorithms, signatures, key sizes, or long-term cryptographic validity would require separate standards-based justification. `TODO:CITATION_NEEDED`

4.4 Audit Trail and Provenance Model

The audit trail represents measurements and governance events as append-oriented events. Measurement events preserve a stable relationship between a canonical record and its hashed payload. Correction and permission events represent changes that should be visible to later verification rather than silently overwriting stored values.

Model D adds provenance and permission reconstruction. Actor and key identifiers are included in events, permission grants and revocations are represented explicitly, and the verifier reconstructs active authorization state over the event stream. This enables checks for unauthorized correction, revoked actor key usage, missing correction reasons, broken provenance, and delayed synchronization.

4.5 Verification Workflow

The verification workflow is model-specific. Model A checks schema consistency and duplicate identifiers. Model B checks event payload integrity and event identifiers. Model C additionally checks previous-hash continuity and block hashes. Model D adds provenance, permission, correction, key-state, and synchronization checks.

The verifier does not assess whether a measurement is environmentally plausible or scientifically correct. It only evaluates whether the stored artifact remains internally consistent with the integrity evidence and governance rules represented by the selected model.

5 Evaluation Results

5.1 Evaluation Overview

This section reports the executed proof-of-concept evaluation for the dataset `openaq_capitals_2025_h2`. The results should be read as a controlled threat-coverage evaluation of integrity mechanisms, not as an environmental analysis of the monitored locations. The experiment compares four integrity models against a fixed set of tampering scenarios and records whether the verifier produced the expected alert code for each applicable model/threat pair.

Table 1 summarizes the dataset preparation used for the evaluation. The preprocessing stage converted raw OpenAQ records into canonical measurements and excluded records with missing measurement values.

Table 1: Dataset preparation summary for `openaq_capitals_2025_h2`.

Item	Value
Input OpenAQ records	116,361
Canonical records after preprocessing	112,973
Dropped records	3,388
Selected monitoring locations	12
Cities	4
Monitoring locations per city	3
Parameters after preprocessing	9

5.2 Compared Integrity Models

Table 2 summarizes the four compared integrity models. The model sequence is designed to isolate the effect of adding an audit trail, then hash-chain continuity, and finally provenance and permission reconstruction.

Table 2: Integrity models compared in the proof-of-concept.

Model	Verification capability
A	Conventional storage with schema and duplicate record identifier checks.
B	Audit trail with event payload and event identifier checks, without hash-chain continuity.
C	Audit trail plus hash chain, adding previous-hash and block-hash continuity checks.
D	Hash chain plus provenance and permissions, adding actor/key, permission, correction, provenance, and delayed-synchronization checks.

5.3 Executed Scenario Matrix

The full implemented scenario matrix generated 25 controlled tampering scenarios. For each scenario, the proof-of-concept produced a tampered artifact, a scenario label, a verification output, and a per-scenario evaluation output. The aggregate status counts are shown in Table 3.

The run produced 20 detected cases and 5 expected non-detections. No scenarios were marked as missed, partial, or unexpected-alert cases. These counts should be interpreted as scenario-level verification outcomes for the controlled benchmark, not as statistical detection rates.

5.4 Threat-Coverage Matrix

Table 4 presents the threat-coverage matrix for Models A-D. The matrix uses three compact status codes: D for detected, END for expected non-detection, and NA for not applicable. A detected cell indicates that the expected alert code was present in the verifier output. An expected non-detection

Table 3: Aggregate scenario status counts.

Status	Count
detected	20
expected_not_detected	5
missed	0
partial	0
unexpected_alert	0

indicates an explicit model limitation rather than an implementation failure. A not-applicable cell indicates that the scenario was outside the implemented scope of the model.

Table 4: Threat-coverage matrix for integrity Models A-D.

Threat	A	B	C	D
value_modification	END	D	D	D
timestamp_modification	END	D	D	D
record_deletion	END	END	D	D
fake_record_insertion	END	D	D	D
replay	D	D	D	D
broken_provenance	NA	NA	NA	D
unauthorized_correction	NA	NA	NA	D
revoked_actor_key_usage	NA	NA	NA	D
missing_correction_reason	NA	NA	NA	D
delayed_synchronization	NA	NA	NA	D

The model-level coverage counts are summarized in Table 5. Not-applicable scenarios are excluded from the applicable-scenario denominator. Expected non-detections are retained as explicit model limitations.

Table 5: Applicable scenario coverage by model.

Model	Applicable	Detected	Expected not detected	Other
A	5	1	4	0
B	5	4	1	0
C	5	5	0	0
D	10	10	0	0

5.5 Model-Level Interpretation

Model A detected only the replay scenario in the implemented matrix because replay introduced a duplicate record identifier. It did not contain independent audit or hash evidence for detecting direct value modification, timestamp modification, record deletion, or fake insertion.

Model B detected value modification, timestamp modification, fake record insertion, and replay through audit-event integrity checks. It did not detect record deletion in this implementation because its audit events were not linked into a hash chain.

Model C detected all five base scenarios. The addition of hash-chain linkage made stream-continuity disruption visible, including record deletion.

Model D detected all Model C scenarios and the five additional provenance, permission, correction, revoked-key, and delayed-synchronization scenarios. This result supports the intended role of Model D as the most expressive model in the proof-of-concept, while remaining limited to the controlled scenarios implemented in this benchmark.

5.6 Reproducibility Artifacts

The primary machine-readable outputs are the scenario metrics CSV file, the threat-coverage matrix CSV file, the metrics summary JSON file, and the experiment-run manifest. The manifest records artifact paths, file sizes, hashes, scenario counts, and run metadata for the executed scenario matrix.

6 Discussion

6.1 Interpretation

The experiment shows a progressive increase in threat coverage as additional integrity mechanisms are introduced. Conventional storage provides only limited evidence for post-hoc verification. Adding an audit trail enables detection of local event payload changes, but without hash-chain continuity it remains limited for deletion and stream-order threats. Adding hash-chain linkage makes continuity violations visible. Adding provenance and permission reconstruction extends verification from structural tampering to actor-related and governance-related failures.

The main implication is that blockchain-inspired design should be evaluated at the level of concrete integrity functions rather than as a general claim that blockchain improves environmental monitoring. In the proof-of-concept, the useful mechanisms are audit events, deterministic hashes, previous-hash linkage, explicit permission events, correction metadata, and verifier workflows.

6.2 Scope of Environmental Interpretation

The OpenAQ dataset provides a realistic distributed environmental monitoring substrate, but the experiment does not produce environmental-domain conclusions. The results do not compare air quality between Warsaw, Berlin, Paris, and Madrid, do not evaluate pollutant behavior, and do not assess sensor calibration or environmental anomaly detection. The selected monitoring locations support a multi-source dataset for information-systems evaluation.

6.3 Limitations

The scenario set is controlled and synthetic. Each scenario uses one deterministic injection per applicable model/threat pair. Therefore, the current evaluation should not be described as a statistical robustness study over many attack positions, stations, timestamps, or random seeds.

The current experiment does not define negative cases. For this reason, precision, recall, F1 score, false-positive rate, and false-negative rate are not reported. The appropriate outputs are scenario status counts, model-level coverage, and the threat-coverage matrix.

The proof-of-concept uses one OpenAQ extract and one global event stream per integrity model. It does not build independent chains per station, city, or gateway. It also does not implement cross-sensor plausibility checking, full intermittent-connectivity simulation, summary-block optimization, or production-grade key management.

6.4 Future Work

Future work can extend the benchmark with repeated injections across multiple records, stations, timestamps, and random seeds. A negative-case benchmark would make precision, recall, false-positive rate, and false-negative rate meaningful. Additional work could also evaluate per-station or per-gateway chains, delayed synchronization under offline queues, summary-block verification, and cross-sensor environmental consistency checks as a separate layer from blockchain-inspired integrity verification.

7 Conclusions

This paper presented a lightweight proof-of-concept for evaluating blockchain-inspired data integrity mechanisms in distributed environmental monitoring systems. The study compared conventional storage, audit trails, audit trails with hash chains, and audit trails with hash chains plus provenance and permission reconstruction.

Using a bounded OpenAQ dataset and controlled tampering scenarios, the evaluation produced a threat-coverage matrix that distinguishes model capabilities from expected limitations. The results show that audit trails, hash-chain continuity, and provenance/permission reconstruction address different classes of integrity threats. Conventional storage provides limited post-hoc integrity evidence; audit trails improve local event verification; hash chains improve stream-continuity verification; and provenance/permission reconstruction supports actor-related and governance-related checks.

The contribution is intentionally limited to information-systems integrity evaluation. The work does not claim environmental outcomes, air-quality findings, production blockchain readiness, or regulatory compliance. Its value lies in providing a reproducible structure for comparing integrity models and verification workflows using realistic environmental monitoring data as the application substrate.